# Architecture Discoverability and Verification

Agata Przybyszewska, PhD

Senior Enterprise Architect · DSV

Architect, DevSecOps engineer, Coder · MathMagicians

Faglig Fredag · 30 October 2026

# Background

# Big enterprise, stale docs

What problem are we trying to solve?

Compliance, audit, governance.

Stop gates - you want it deployed? You need approval.

Then … life goes on[1] …




1. *The author believes that the thing worse than no documentation, is stale documentation.*

# Must architecture docs always go stale?

# So what is Architecture Discoverability?

Three questions

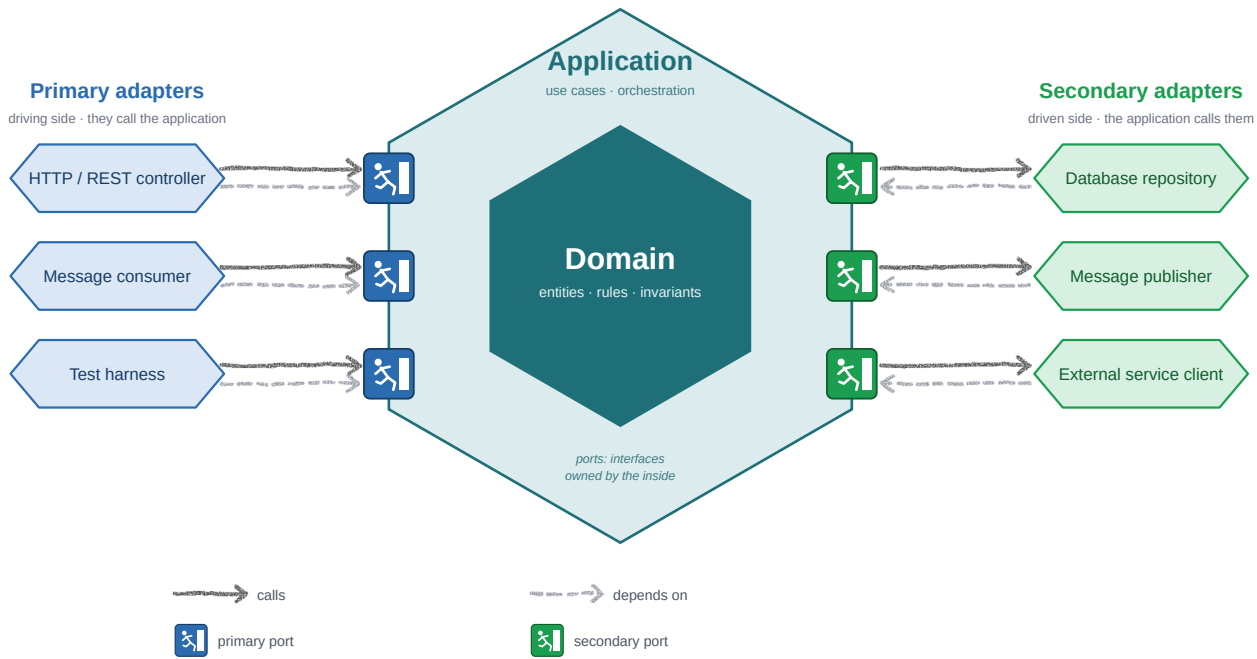
How do we specify it?

How do we keep it in sync, as life goes on?

How do we verify it?

# Hexagonal Architecture

This style will be used in our examples.



# Meet Wonderland, a toy platform

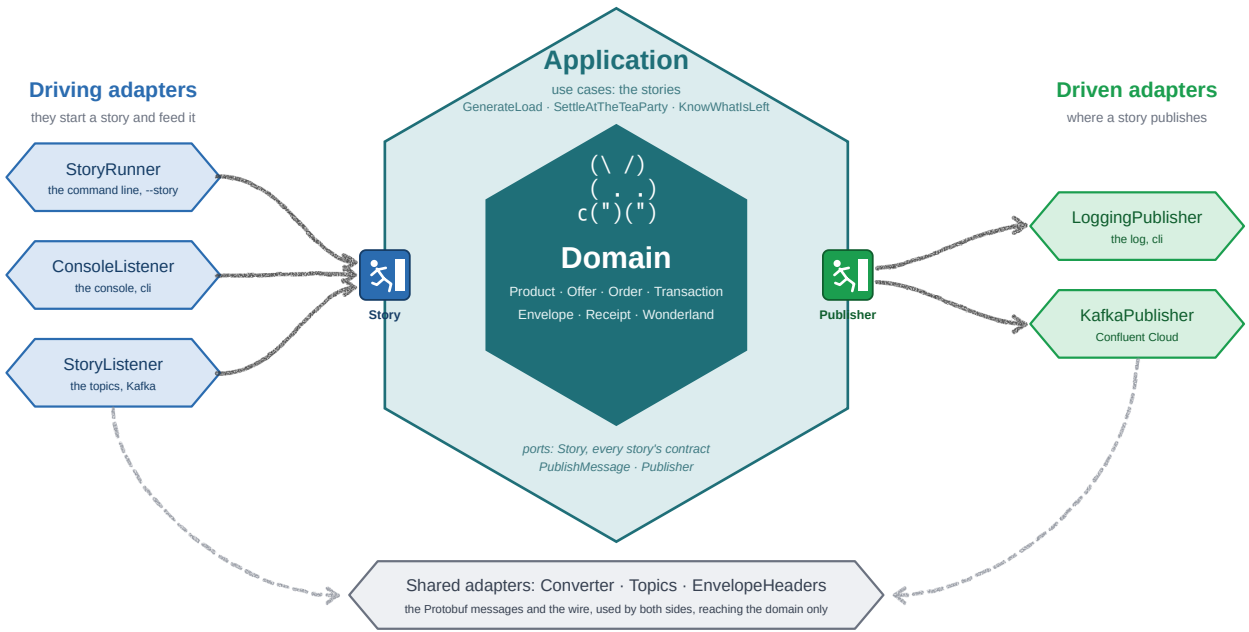
Built to explore the core ideas

Platform Engineering is a skill

DX, make it easy - red carpet to do the right thing

Keep it simple, hide away complexity and deep tech stacks

# Wonderland, as planned



# Wonderland - Tech stack

Why Java? Enterprise

*μ-services*: Java 25, Spring Boot, etc ...

*Runtime:* JRE, Docker + Docker Compose to run it

*Testing:* TDD: JUnit, BDD: Cucumber, Testcontainers

*Eventing Platforms:* Kafka, Confluent[1] , Cloudflare, CLI

*IaC:* Terraform, Terraform Cloud

*SDLC:* GitHub Actions, Makefile, Gradle[2]

*Architecture discovery:* ArchUnit, JMolecules, PlantUML + our PlantUML plugin

1. *expensive*
2. *... and the slides you see run from a cosy Docker container with Slidev*

# Java 25, Spring Boot, Docker

μ-services: Java 25, Spring Boot + Runtime: JRE, Docker + Docker Compose

| | | Run |
|---|---|---|
| **Java 25 + Spring Boot**<br>EventingPlatform.java | The Spring Boot application is the substrate for the ports and the adapters; a process plays one story | `make run ARGS="--story=tea-party"` |
| **Docker + Compose**<br>compose.yaml | Starts the flock from one image: a generator per region and the tea party | `TTL=300 make up` |

# Development methodology

Testing: TDD: JUnit, BDD: Cucumber, Testcontainers

| | | Run |
|---|---|---|
| **TDD**<br>TransactionTest.java · ArchitectureTest.java | Unit tests with JUnit 5, AssertJ, and Mockito, one class per production class; the architecture rules are unit tests too | `make test` |
| **BDD**<br>i_can_generate_events.feature · StreamSteps.java | Cucumber features and step definitions; each feature plays against the image in Testcontainers, a red feature blocks the build | `make bdd` |
| **Spec Driven**<br>How we work · discovery.md | Discovery → Formulation → Automation → Verification; a release plan per folder, the board follows it | `make gh-project-sync` |

# Kafka, Confluent, CLI

Eventing Platforms: Kafka, Confluent, Cloudflare, CLI

| | | Run |
|---|---|---|
| **Kafka on Confluent Cloud**<br>KafkaPublisher.java · kafka.properties | Records carry CloudEvents headers and a Protobuf payload; the key picks the partition | `make run-tiny TRANSPORT=confluent` |
| **CLI**<br>ConsoleListener.java | Without a cluster: a story publishes to the log and hears what is typed on stdin | `make run ARGS="--story=tea-party --tea-party.ttl="` |

# The DevSecOps part

IaC: Terraform, Terraform Cloud + SDLC: GitHub Actions, Makefile, Gradle

| | | Run |
|---|---|---|
| **Terraform + Terraform Cloud**<br>topics.tf | Topics, schemas, and Flink tables, applied on every push to `main` | `make tf-plan` |
| **GitHub Actions**<br>cicd.yaml | Makefile drives the GitHub workflow, commands can be run locally | `make check` |
| **Makefile**<br>Makefile | All commands work locally, and drive the CICD workflows. | `make help` |
| **Gradle**<br>platform.gradle | `platform.gradle` holds the conventions every build shares; a story's own build applies it too | `make build` |

# jMolecules + ArchUnit

Architecture discovery: ArchUnit, JMolecules, PlantUML + our PlantUML plugin

jMolecules stereotypes on ports and adapters; one ArchUnit rule fails the build when the core reaches an adapter

```
make arch-verify
```

HexagonalArchitecture.java

# Generate architecture diagrams from code

Architecture discovery: ArchUnit, JMolecules, PlantUML + our PlantUML plugin

Generates the hexagon from the code; `make check` fails when the drawing is not regenerated

```
make arch-gen
make arch-check
```

architecture-discoverability.md

# Wonderland plays stories that send messages to each other

Kafka, Confluent, and CLI are the transports; the stories are the same on each

# First iteration

cli, confluent

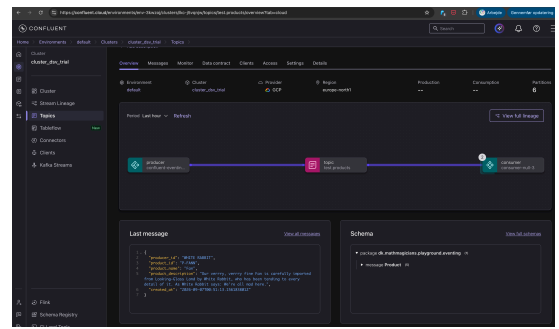
**cli**: the console stands in for the topics, the log is where a story publishes, stdin is what it hears

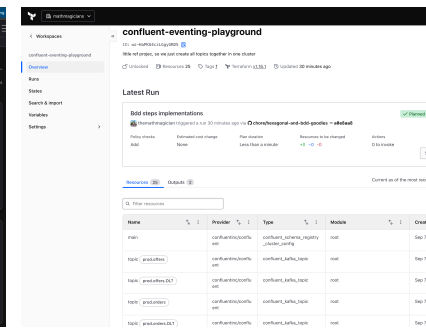
```
make run ARGS="--story=tea-party --tea-party.ttl="
Offer offer_id: "OFF-1" product_id: "P-TOPH" price: 12.5 sel
Order id: "ORD-1" customer_id: "ALICE" product_id: "P-TOPH"
```

```
12:00:03.117  INFO [main] SettleAtTheTeaParty : tea-party: S
12:00:03.121  INFO [main] LoggingPublisher : Published 7c2e-
```

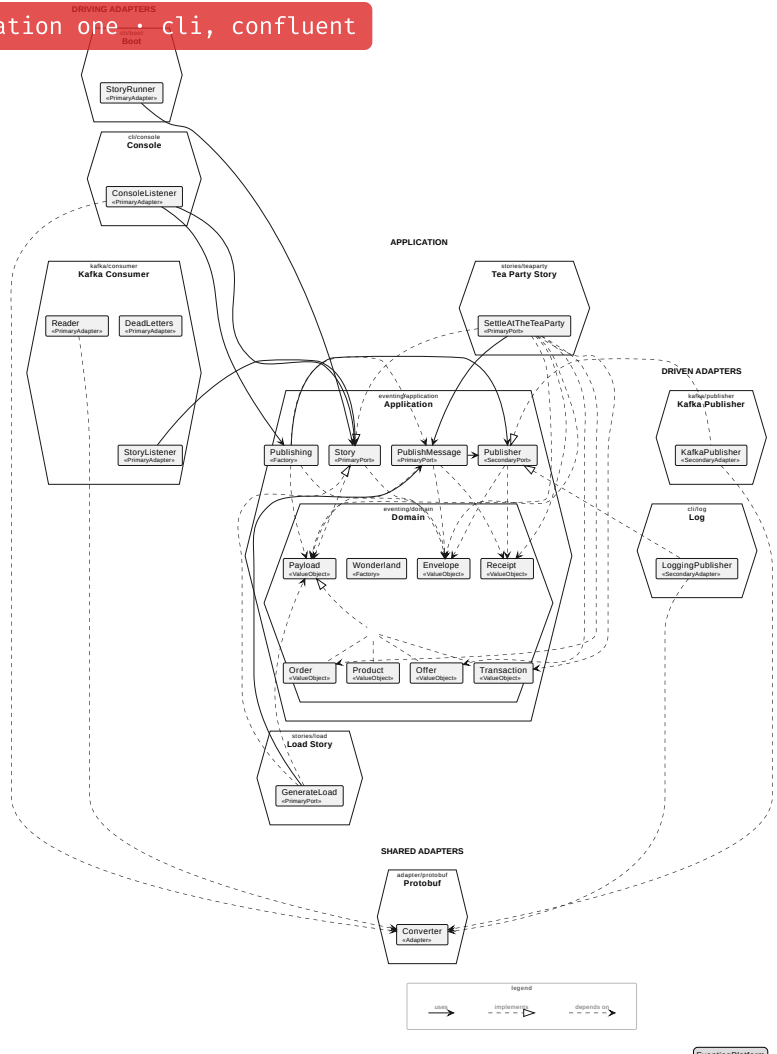
**confluent**: Confluent Cloud, the topics, schemas, and Flink tables from `iac/`, applied by Terraform Cloud; the keys from `.env.test.private`

```
make tf-plan
make run-tiny TRANSPORT=confluent
```





● REC · generated from the code · iteration one · cli, confluent



# The wiring: a transport is a profile

A process plays `<transport>,<env>`; the adapters of a transport carry it, the properties of a transport connect it

```
@PrimaryAdapter @Component @Profile("cli")        public final class ConsoleListener ...    // adapter/cli/console
@SecondaryAdapter @Component @Profile("cli")      final class LoggingPublisher ...          // adapter/cli/log
@PrimaryAdapter @Component @Profile("confluent")  public final class StoryListener ...      // adapter/kafka/consumer
@SecondaryAdapter @Component @Profile("confluent") final class KafkaPublisher ...          // adapter/kafka/publisher
```

```
# platform.properties: the transport when none is given
spring.profiles.default=cli
# application-confluent.properties: the cluster, the registry, SASL, the serializers; a missing key fails startup
spring.config.import=classpath:kafka.properties
spring.kafka.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS}
```

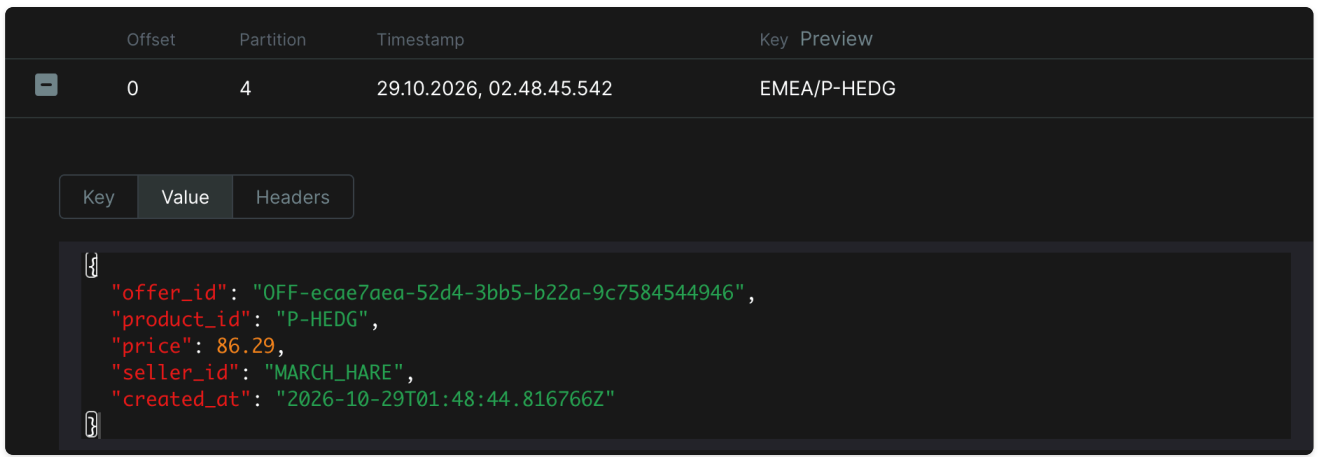
```
make run TRANSPORT=confluent ENV=test    # SPRING_PROFILES_ACTIVE=confluent,test, the keys from .env.test.private
```

Out of trial credits :)

# The kafka universe

A broker, a schema registry, a topic viewer, and the topics and schemas `iac/` gives Confluent Cloud, in `compose.kafka.yaml`

```
make kafka-universe-up
make run-tiny TRANSPORT=kafka
open http://localhost:8082
```



# The refactoring: kafka is the transport, confluent is kafka with credentials

Five annotations, one group, the defaults for a broker next door

```
@Profile("kafka")   // was "confluent": KafkaPublisher, StoryListener, Reader, DeadLetters, Topics
```

```
# platform.properties: confluent plays kafka's adapters and settings underneath
spring.profiles.group.confluent=kafka
# kafka.properties: the broker and the registry of make kafka-universe-up, unless the environment names others
spring.kafka.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
spring.kafka.properties.schema.registry.url=${SCHEMA_REGISTRY_REST_ENDPOINT:http://localhost:8081}
# application-confluent.properties: what Confluent Cloud adds, SASL and the registry's basic auth
spring.kafka.security.protocol=SASL_SSL
```

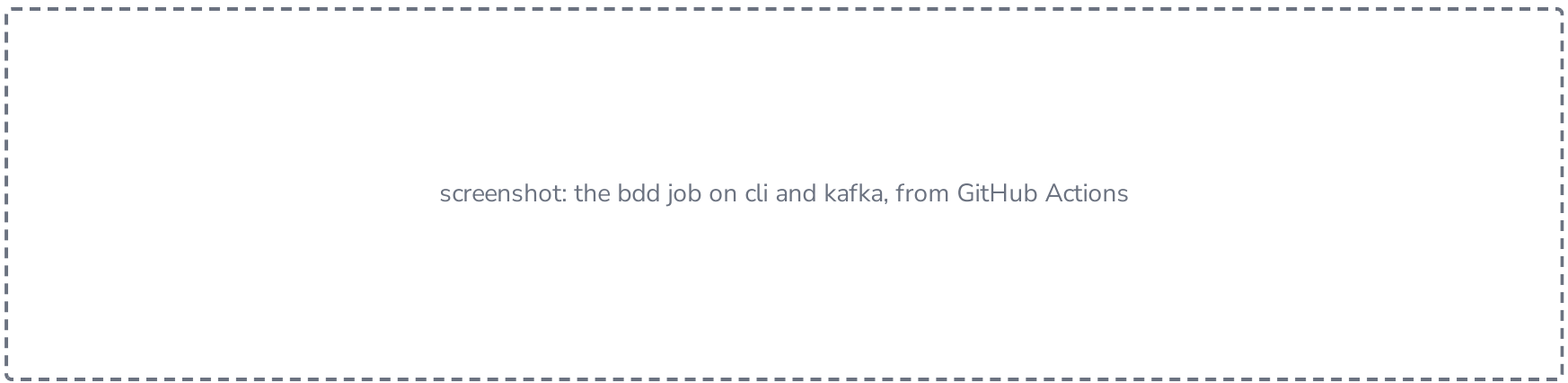
```
make run TRANSPORT=kafka    # SPRING_PROFILES_ACTIVE=kafka,test, no keys; compose.kafka.yaml is what iac/ is for the cloud
```

# The spec on both transports: test + CI

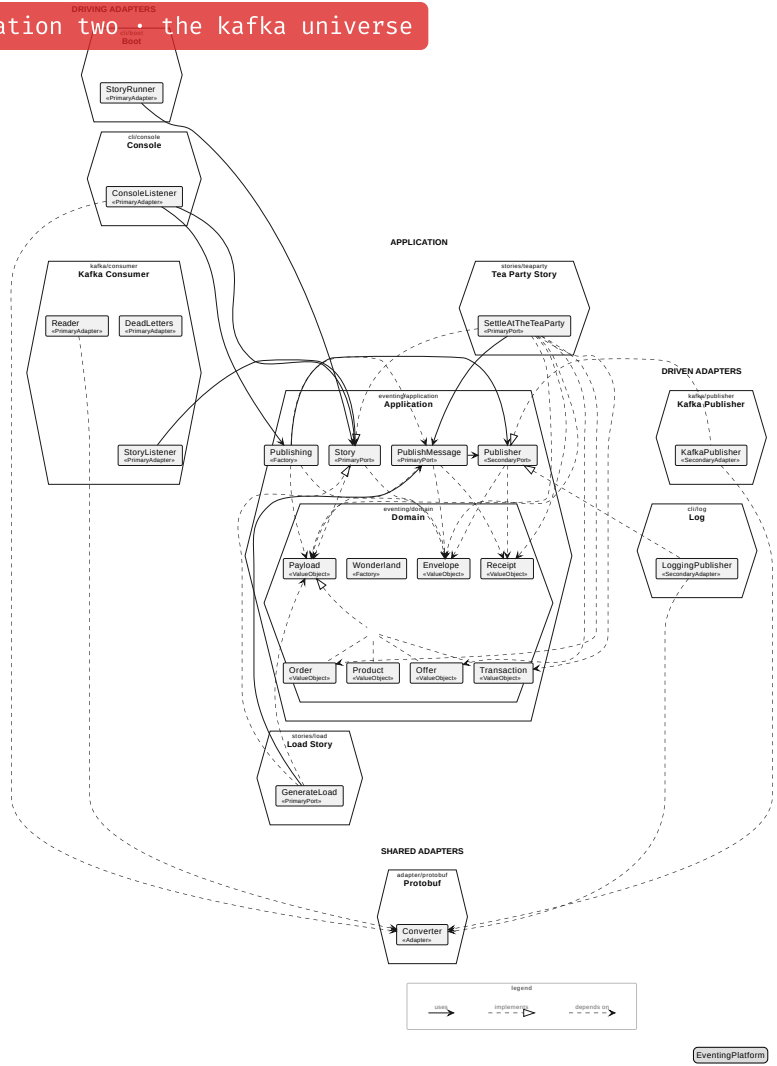
The same features, the same command, once per transport of the matrix

```
make ci-transports            # ["cli","kafka"]; confluent sits beside it, commented out
make bdd                      # cli: the log and the console
make bdd TRANSPORT=kafka      # the universe up first, the containers on its network
make bdd-matrix               # both, locally what the bdd job plays
```

`cicd.yaml` , job `bdd` : `make bdd-published TRANSPORT=<t>` against the candidate image, one job per entry in `<t>-test` ; `cd` publishes what both passed



● REC · generated from the code · iteration two · the kafka universe



# 1 · Declare it in the code

# The annotation is the spec

The platform's driving port, `Story`

```
@PrimaryPort
public interface Story {

    String name();

    default Set<Class<? extends Payload>> listensTo() {
        return Set.of();
    }
}
```

Its twin in the purse, a client's story, `KnowWhatIsLeft`

```
@PrimaryPort
public final class KnowWhatIsLeft implements Story {

    public static final String NAME = "purse";
```

# 2 · Verify it

# Two rules, plain unit tests

# Live: break the hexagon

# 3 · Clients get the checks for free

# The purse extends the rules

# The purse reuses the steps

# 4 · One model, three levels

# Code in, model out

# Three levels

# The model

# Facts found, views chosen

# Level 1 · The forest

# Level 2 · The trees

# Level 3 · The hexagon

# Where the facts live

# Same stories, three transports

# 5 · The agent round trip

# What the agent gets

# The skill: generate a platform client

# Live: an agent builds a new purse

# Gates the agent can’t argue with

# Back in the model

# Three answers

Thanks! Questions?